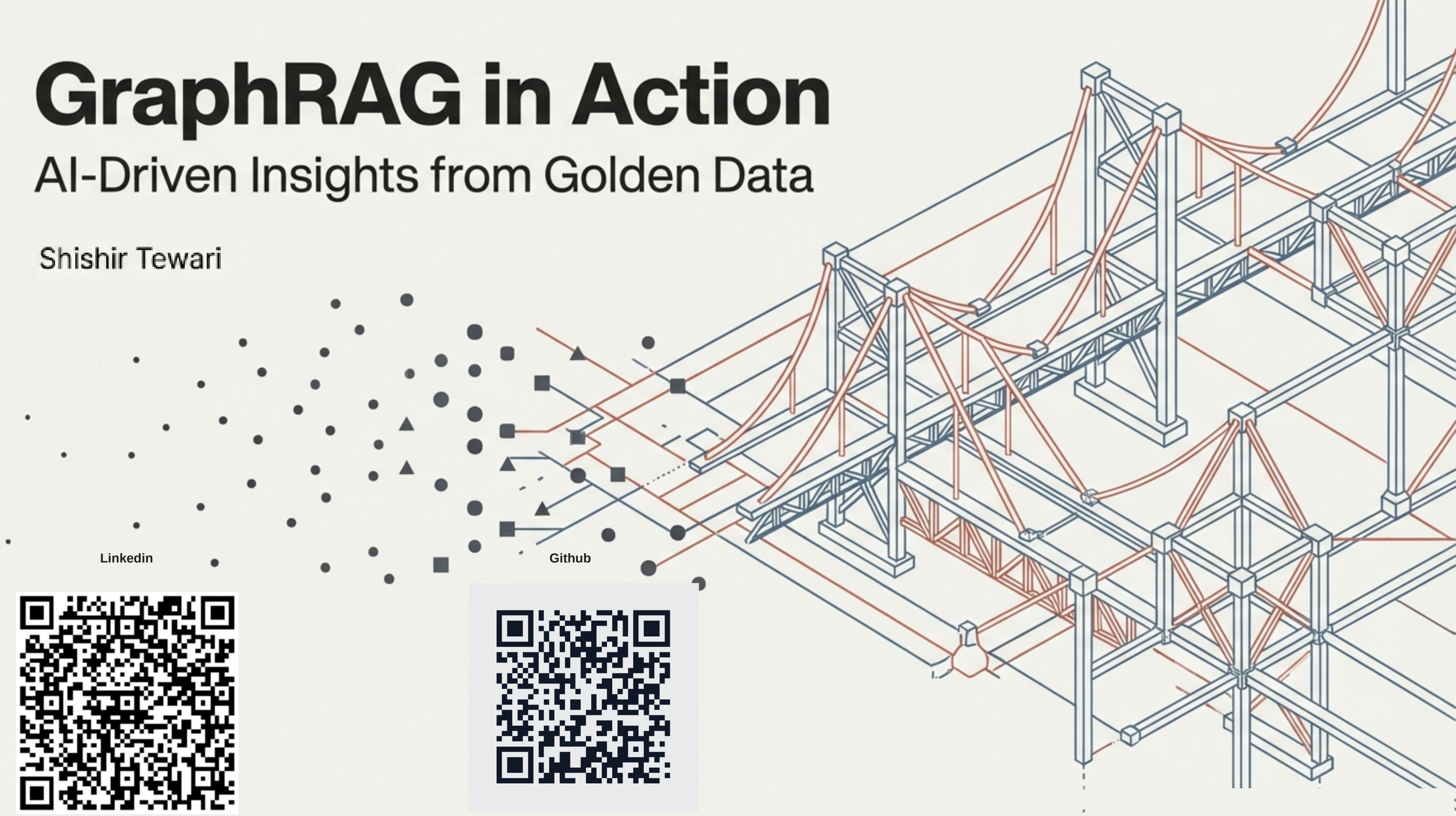
GraphRAG in Action

AI-Driven Insights from Golden Data

Shishir Tewari

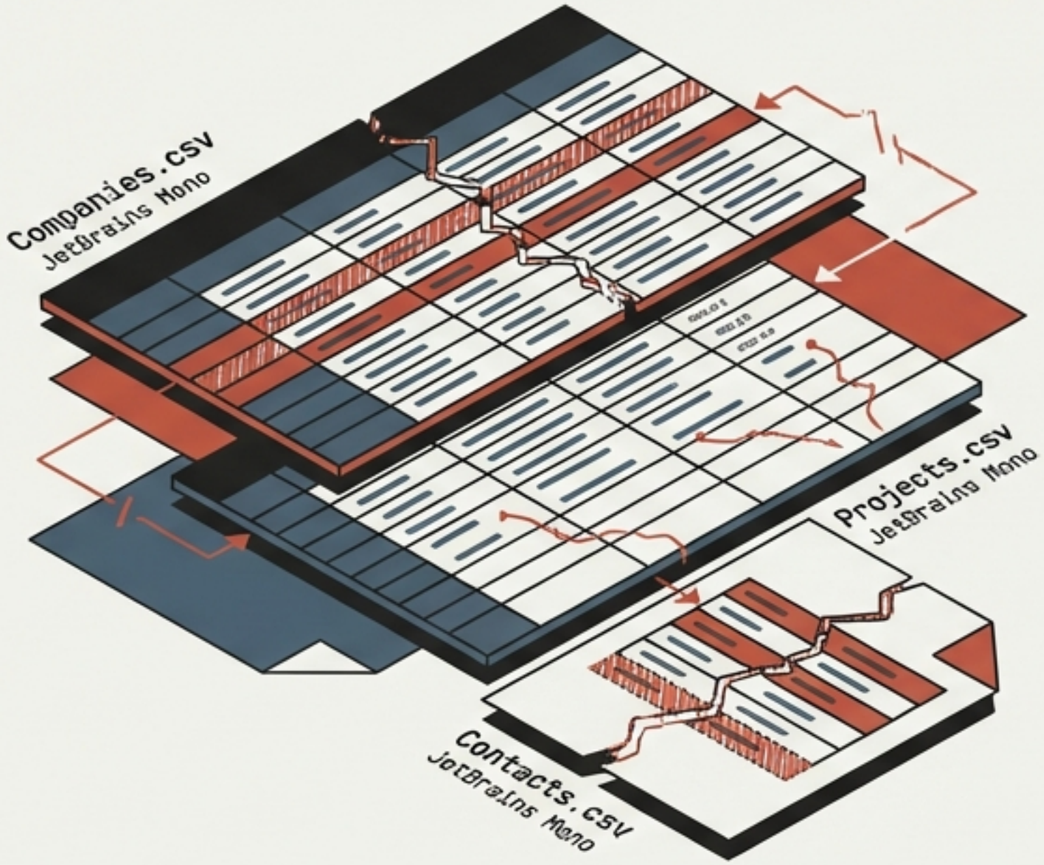


Linkedin

Github

The Reality

Raw data is inherently chaotic: Duplicates, disjointed tables, and fragmented records.



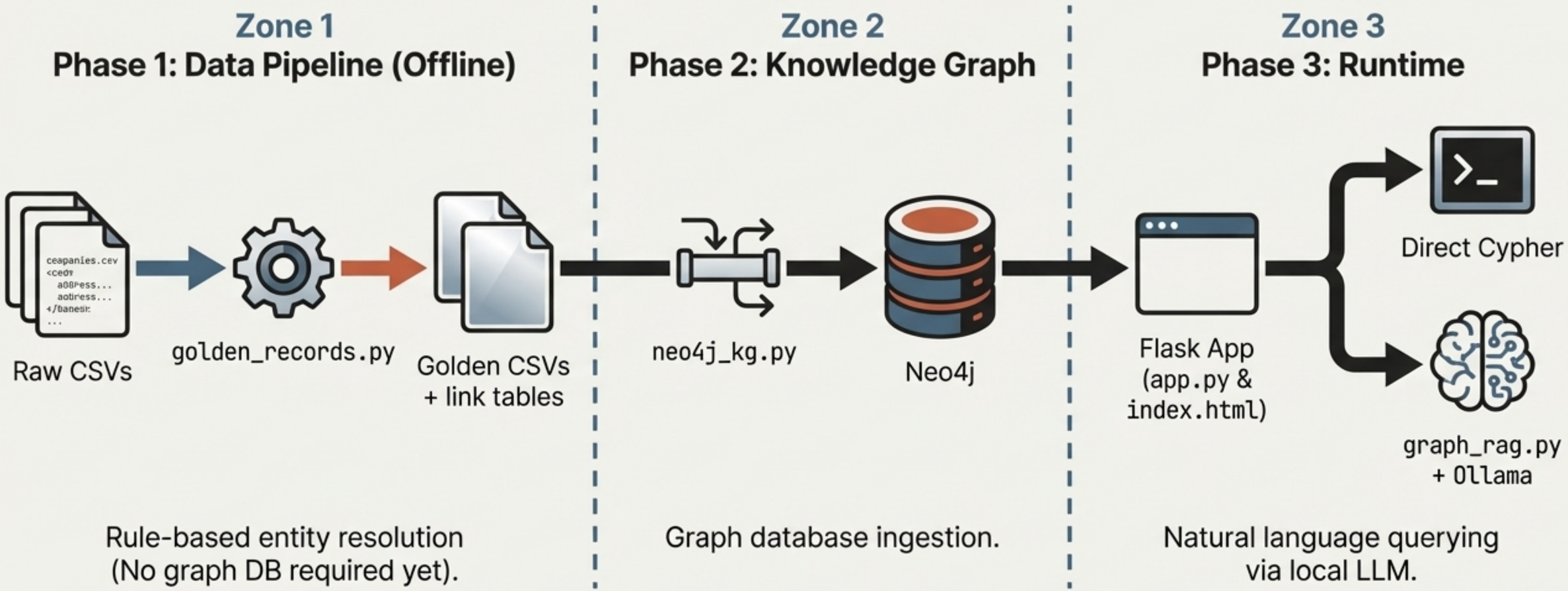
The Expectation

Which companies and contacts have at least two projects?

End users expect frictionless, plain-English answers.

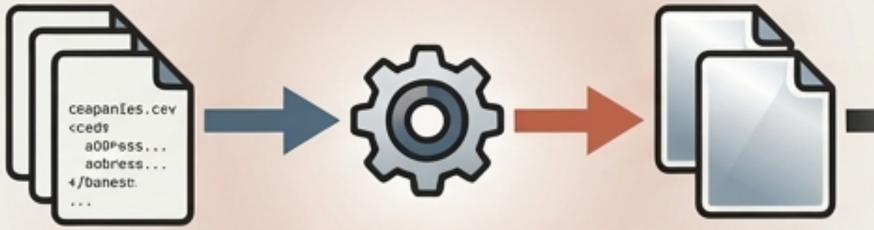
Relational databases struggle to bridge this semantic gap natively.

Mapping the journey from raw data to actionable AI insights



Isolating the offline data pipeline for modularity and scale

Zone 1
Phase 1: Data Pipeline (Offline)



Rule-based entity resolution
(No graph DB required yet).

Zone 2
Phase 2: Knowledge Graph

Graph database ingestion.

Zone 3
Phase 3: Runtime

- Executes entirely offline.
- Decoupled from the database layer for easier testing and logic updates.
- Output: Clean entities and link tables

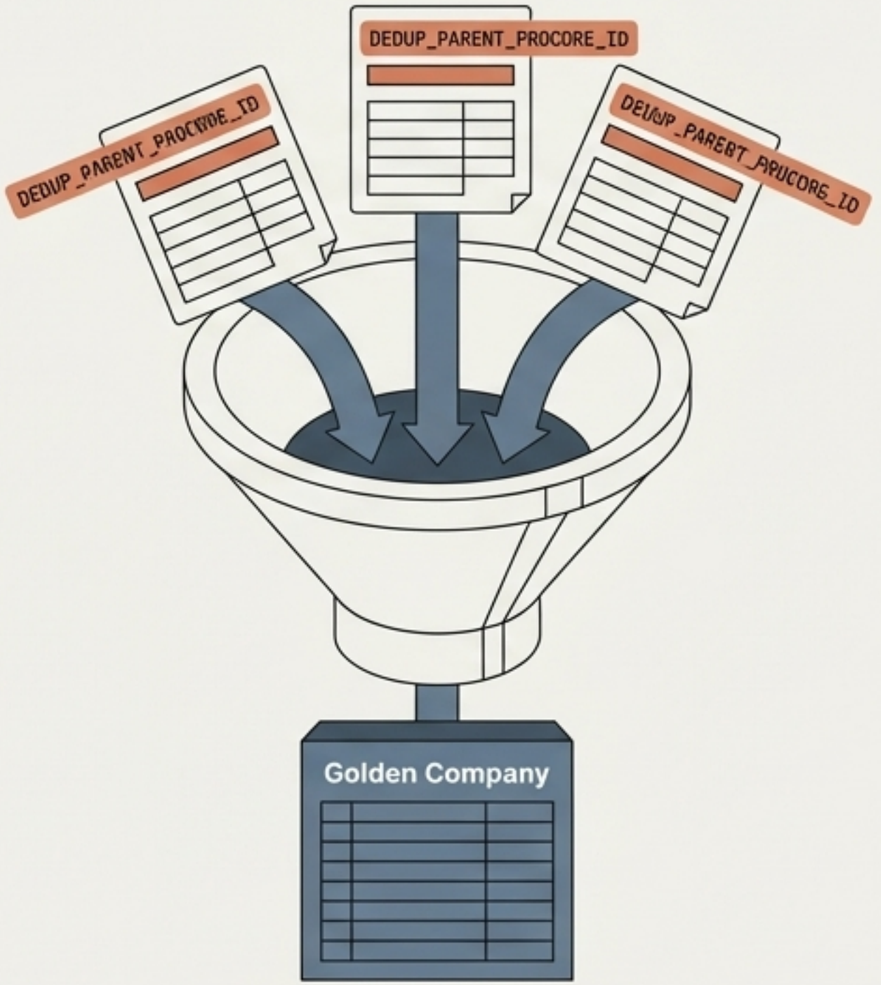
`company_project_links.csv`
`company_contact_links.csv`

Natural language querying
via local LLM.

Forging single sources of truth from clustered entities

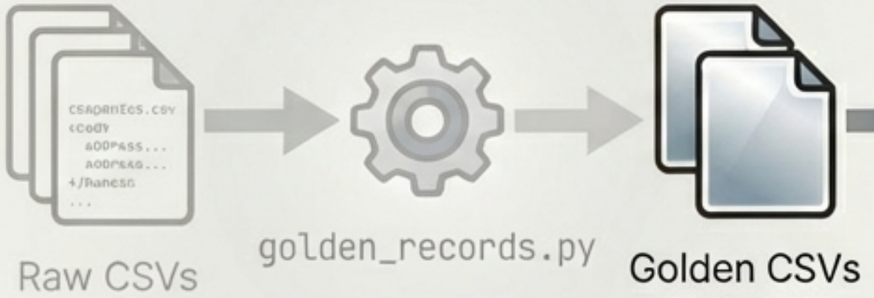
- **Companies**: Clustered via `DEDUP_PARENT_PROCORE_ID`. Pipeline prefers the canonical row but merges non-null fields (like `ESTIMATED_ACV`) from the entire cluster.
- **Mapping:** `company_id_src` / `COMPANY_PROCORE_ID` map directly to the newly forged `company_cluster_id`.
- **Projects & Contacts:** Re-mapped to link exclusively to Golden Companies.

Note: Currently rule-based, designed to easily integrate LLMs for future disambiguation.



Translating flat golden records into an interconnected graph

Zone 1
Phase 1: Data Pipeline (Offline)



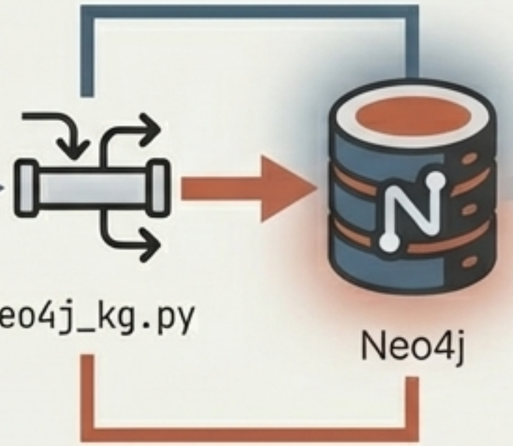
Raw CSVs

`golden_records.py`

Golden CSVs
+ link tables

Rule-based entity resolution
(No graph DB required yet).

Zone 2
Phase 2: Knowledge Graph
(Scaled up by 20%)



`neo4j_kg.py`

Neo4j

Graph database ingestion.

Zone 3
Phase 3: Runtime

Graph databases bridge the gap for LLMs by storing data exactly how humans describe it.

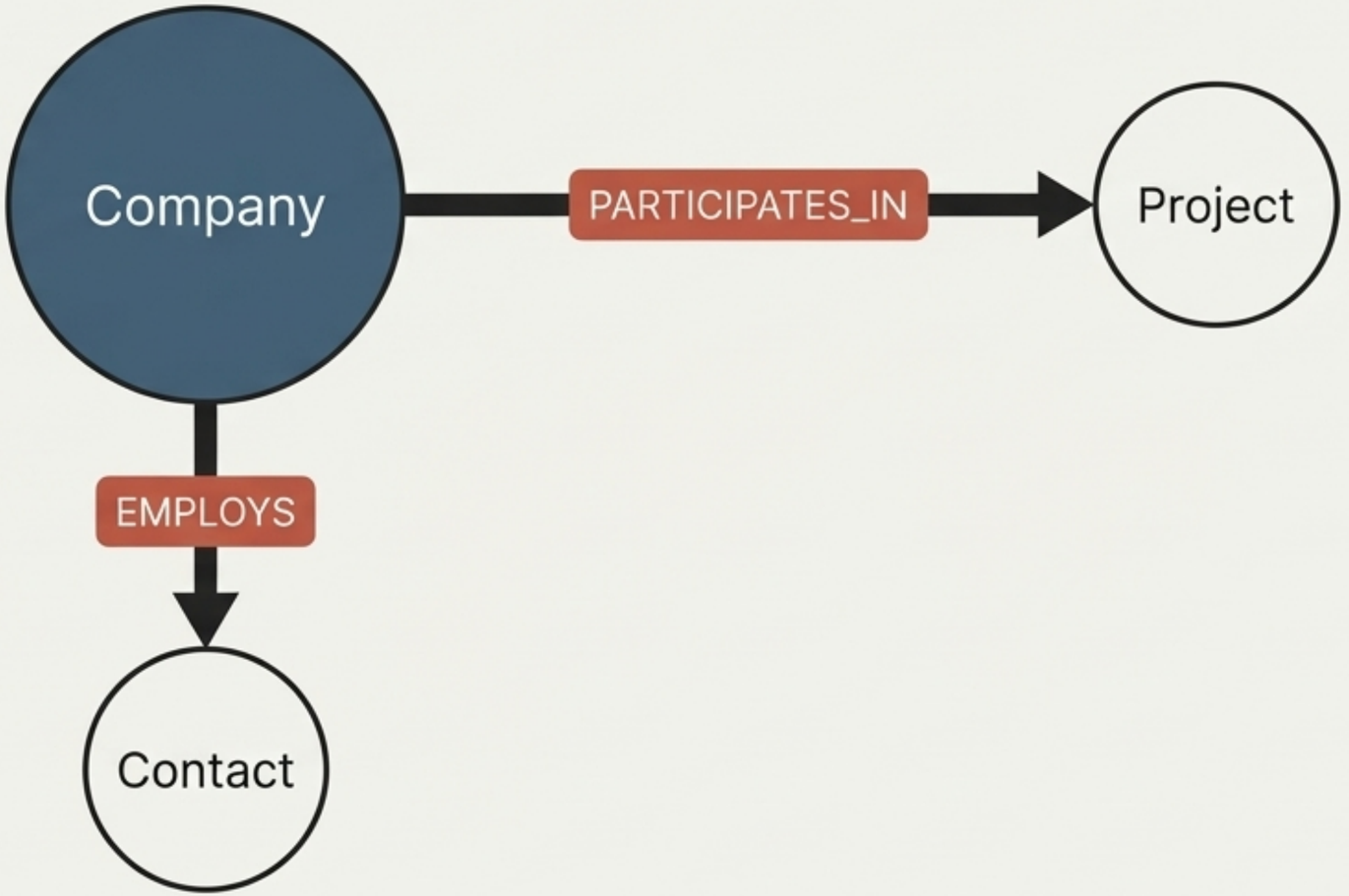
`neo4j_kg.py` reads the `golden/` directory and executes the database load.

Clears and reloads Neo4j by default (supports `--no-clear` for appending).

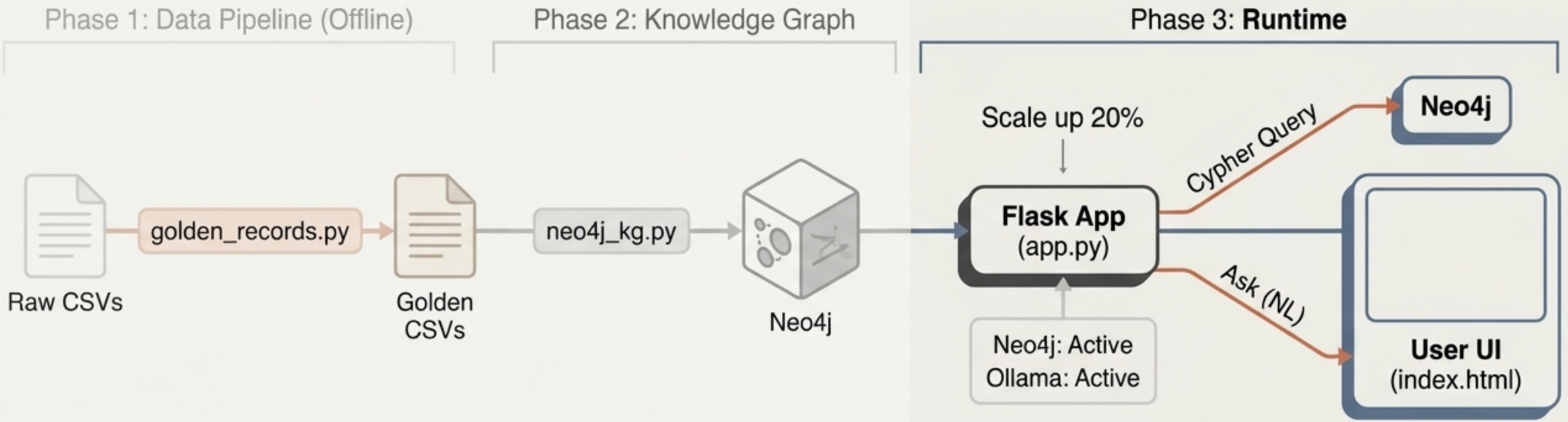
Natural language querying
via local LLM.

Structuring the data exactly how human beings think about it

- **Nodes:** Company, Project, Contact.
- **Relationships:** Driven entirely by the link tables generated in the offline pipeline.
- **Future Extensibility:** The schema is primed for a Contact ASSIGNED_TO Project relationship when assignment data becomes available.



Executing the runtime query flow

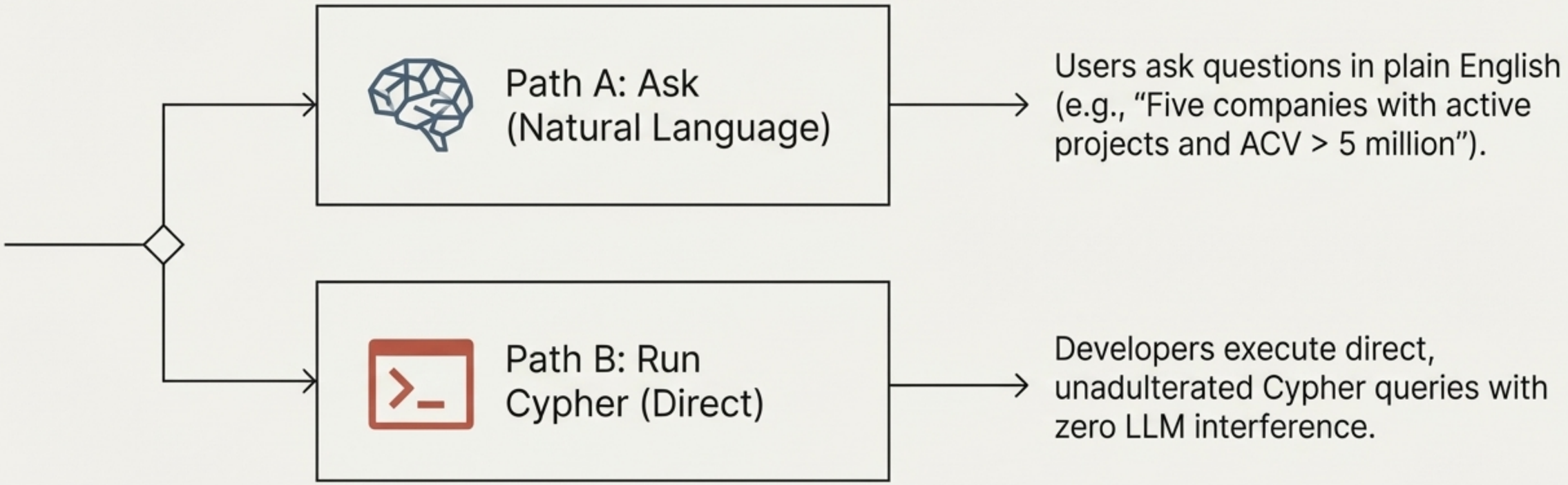


Phase 1: Data Pipeline (Offline)
previous crange flow of Raw CSVs throug golden_records.py and grayscine to Golden CSVs.

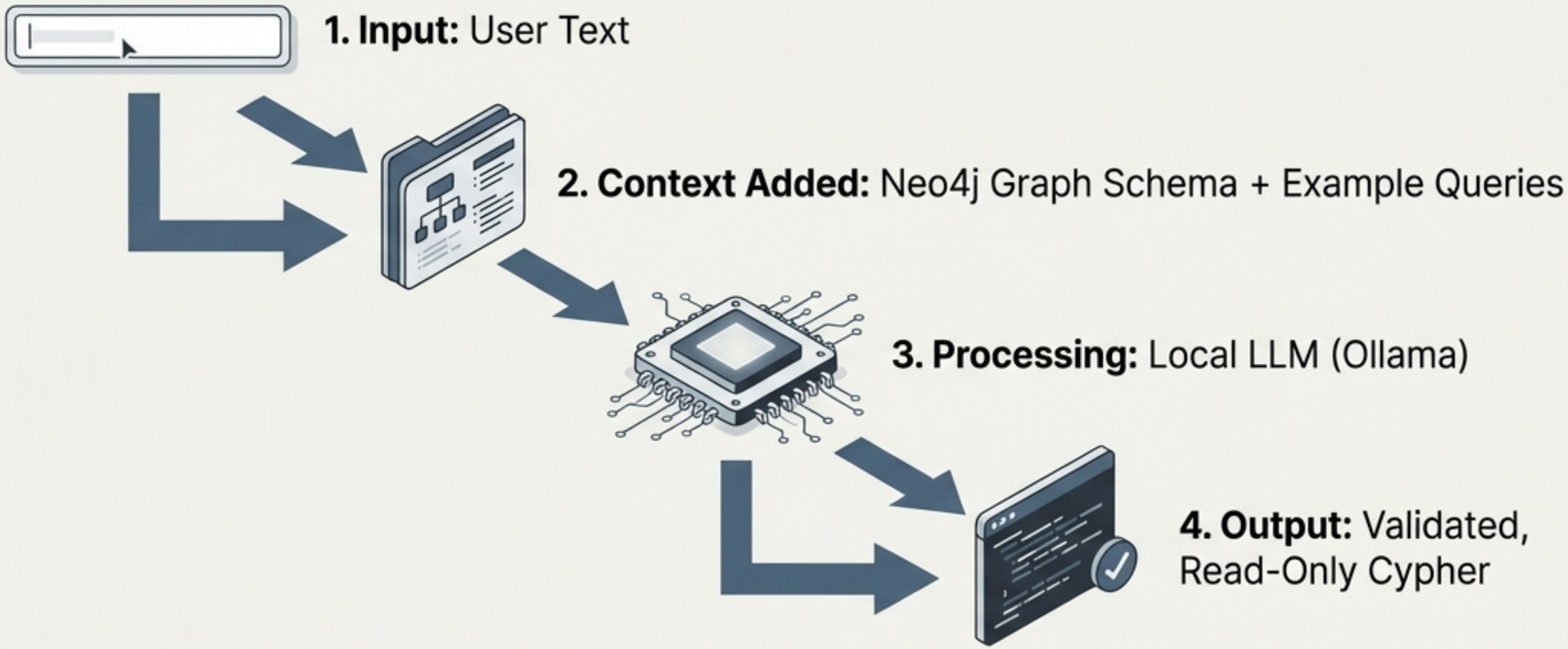
Phase 2: Knowledge Graph
previous gravene flow via neo4j_kg.py via Neo4j icon Neo4j database and the local Ollama instance.

- A lightweight Flask application (`app.py`) serving a web UI (`index.html`).
- Exposes endpoints: `/api/query`, `/api/ask`, `/api/graph`.
- Provides live connection status for both the Neo4j database and the local Ollama instance.

Providing dual access for business users and developers



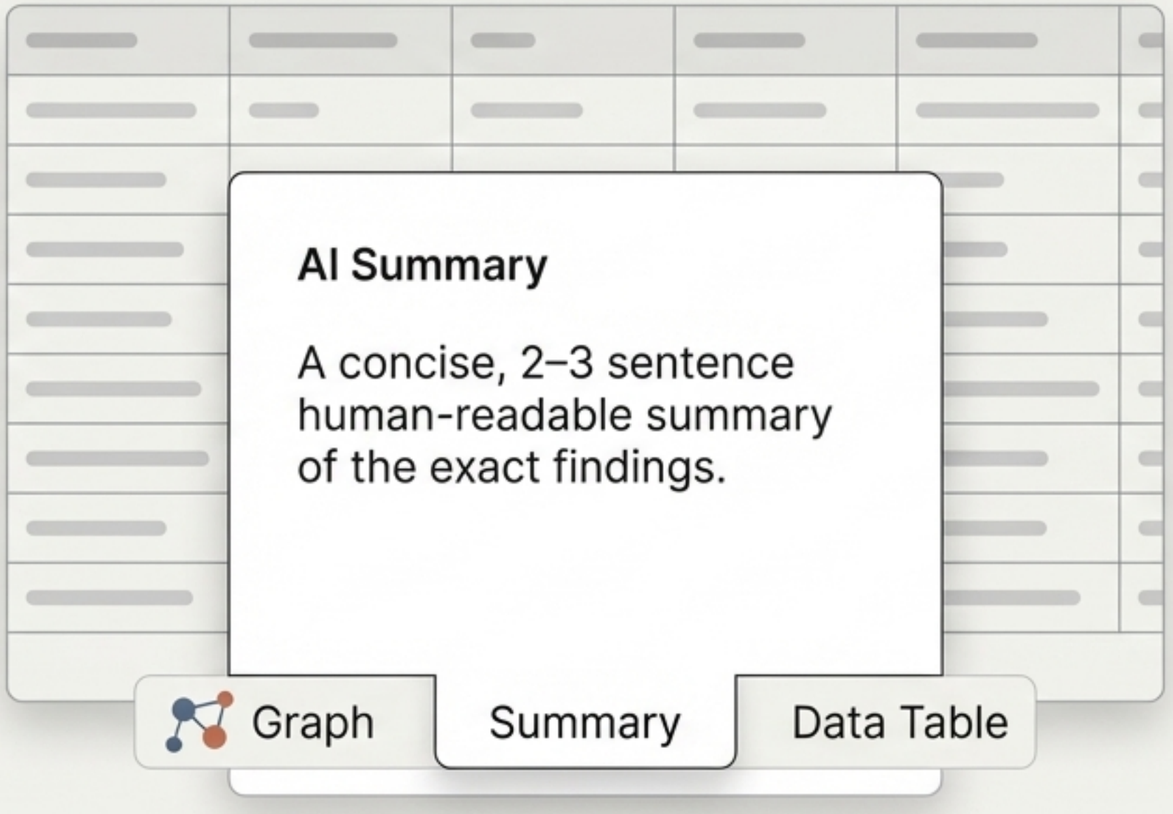
Translating natural language into validated Cypher queries



- Providing the exact schema drastically reduces LLM hallucinations compared to standard text-based RAG.
- The LLM is constrained to generate read-only Cypher, ensuring database integrity.
- The generated Cypher is exposed in the UI, allowing users to learn, edit, and re-run the query.

Delivering context instead of just rows and columns

- Neo4j executes the validated Cypher and returns the raw result.
- app.py passes the result back to Ollama.
- Ollama generates a concise, 2–3 sentence human-readable summary of the exact findings.
- The user receives the Summary, the Data Table, and the Graph visualization simultaneously.



Mapping the components and deployment stack

Configuration	`config.py` manages paths, credentials, and hosts.
Database	Compatible with local Neo4j Desktop (`bolt://127.0.0.1:7687`) or Cloud (Neo4j Aura Free).
LLM Runtime	Local Ollama execution (default: llama3.2 on `http://localhost:11434`).
Privacy	Running the LLM locally ensures proprietary corporate data never leaves the environment.

Executing the pipeline from raw data to running application

1. Build Golden Records

```
python golden_records.py
```

2. Load the Knowledge Graph

```
python neo4j_kg.py
```

3. Start the Local LLM

```
ollama pull llama3.2
```

4. Launch the Application

```
flask --app app run
```

True artificial intelligence requires foundational data structure

- GraphRAG bridges the gap between messy corporate data and actionable, natural language insights.
- By refining data offline and mapping it relationally, we eliminate hallucinations and unlock true analytical power.

Shishir Tewari